

Exercícios

1. O objetivo deste exercício é compreender como a **herança**. Utilizaremos o exemplo de uma hierarquia acadêmica, onde **Aluno** e **Professor** herdam características de **Pessoa**.

Superclasse (Pessoa)

Desenvolva a classe **Pessoa** como a base da hierarquia, contendo os seguintes atributos privados:

Identificador	Tipo	Descrição
nome	String	Nome da pessoa
idade	int	Idade da pessoa

A classe deve conter métodos **get** para os atributos privados e o seguinte construtor público:

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, idade	Inicializa os atributos da superclasse

Subclasses (Aluno e Professor)

Implemente as subclasses utilizando a palavra-chave **extends**:

- **Classe Aluno:** Adiciona o atributo privado **curso** (**String**). Utilize **super(nome, idade)** no construtor para inicializar os atributos herdados.

Identificador	Tipo	Descrição
curso	String	Curso em que o aluno está matriculado

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, idade, curso	Inicializa atributos da superclasse e o curso
toString	String	–	Retorna representação textual formatada do aluno

- **Classe Professor:** Adiciona o atributo privado **matricula** (**int**). Utilize **super** para delegar a inicialização dos dados herdados.

Identificador	Tipo	Descrição
matricula	int	Número de matrícula do professor

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, idade, matrícula	Inicializa atributos da superclasse e a matrícula
toString	String	–	Retorna representação textual formatada do professor

Classe Principal

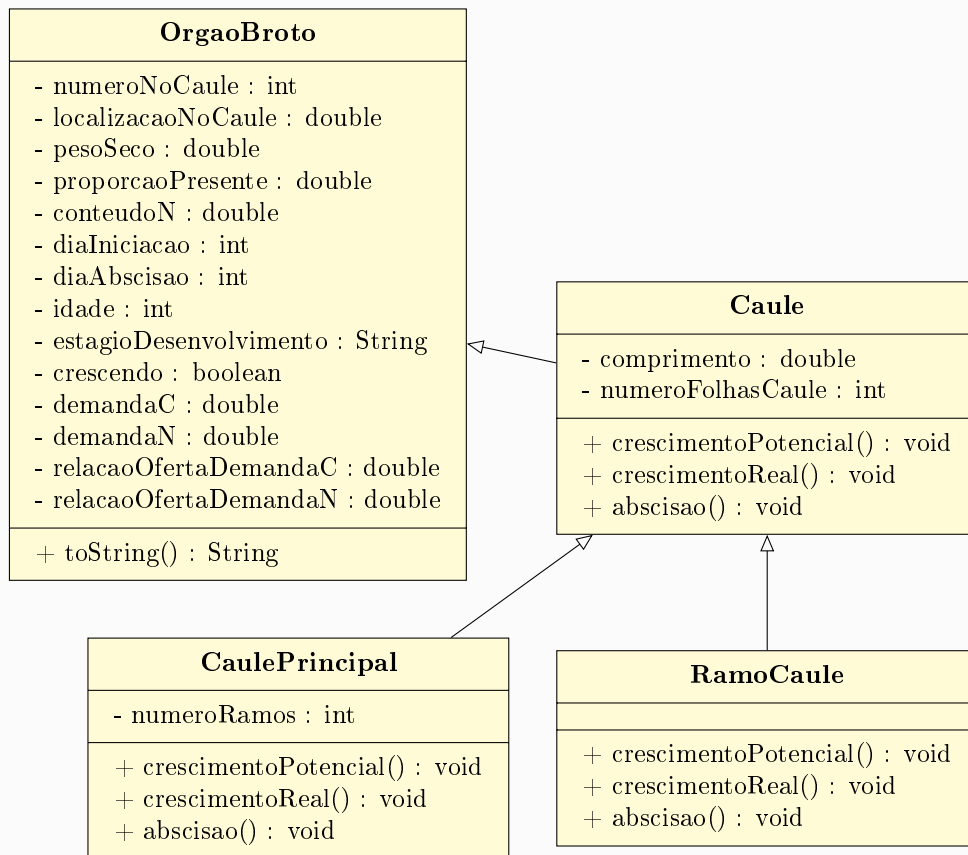
Desenvolva a classe `Classe_aluno_heranca` com o método `main` para instanciar os objetos:

- Crie um objeto `Aluno` e um objeto `Professor`.
- Exiba os dados de cada um utilizando os métodos herdados da classe `Pessoa` (como `getNome()` e `getIdade()`).
- Observe como os dados do aluno incluem o `curso`, enquanto os do professor seguem sua estrutura própria.

Reflexão

Note que, embora `Aluno` e `Professor` sejam classes distintas, ambos compartilham a mesma lógica de armazenamento de nome e idade definida em `Pessoa`. Como você poderia impedir que alguém criasse um objeto do tipo `Pessoa` diretamente, tornando-a uma classe puramente de modelo?

- Considere o diagrama UML de uma hierarquia voltada para a modelagem de órgãos vegetais em um sistema agrícola. Desenvolva as classes com os atributos e métodos traduzidos conforme as especificações a seguir.



Superclasse (OrgaoBroto / *ShootOrgan*)

Desenvolva a classe base contendo os seguintes atributos privados (representando dados de desenvolvimento e demanda):

Identificador	Tipo	Descrição
numeroNoCaule	int	Número do caule associado
localizacaoNoCaule	double	Localização no caule
pesoSeco	double	Peso seco do órgão
proporcaoPresente	double	Proporção presente do órgão
conteudoN	double	Conteúdo de nitrogênio (N)
diaIniciacao	int	Dia de iniciação do órgão
diaAbscisao	int	Dia de abscisão (queda)
idade	int	Idade do órgão em dias
estagioDesenvolvimento	String	Estágio atual de desenvolvimento
crescendo	boolean	Indicador se está em crescimento
demandaC	double	Demanda de carbono (C)
demandaN	double	Demanda de nitrogênio (N)
relacaoOfertaDemandaC	double	Relação de oferta e demanda de carbono
relacaoOfertaDemandaN	double	Relação de oferta e demanda de nitrogênio

O construtor da classe deve inicializar os seguintes atributos:

Identificador	Retorno	Parâmetro	Descrição
construtor	–	numeroNoCaule, diaIniciacao, estagioDesenvolvimento	Inicializa atributos da classe

A classe deve conter métodos **get** e **set** para os atributos e o seguinte método público:

Identificador	Retorno	Parâmetro	Descrição
toString	String	–	Retorna a representação textual dos dados do órgão

Superclasse intermediária (Caule / *Stem*) e Subclasses

Implemente a classe **Caule** que herda de **OrgaoBroto**, adicionando atributos específicos de caule, e crie as especializações **CaulePrincipal** e **RamoCaule**:

- **Classe Caule:** Adiciona os atributos privados **comprimento** (double) e **numeroFolhasCaule** (int). Utilize **super** no construtor.
- **Classe CaulePrincipal (estende Caule):** Adiciona o atributo **numeroRamos** (int) e implementa os comportamentos de crescimento.
- **Classe RamoCaule (estende Caule):** Herda a estrutura de **Caule** e implementa os métodos específicos de crescimento e abscisão.

Dica

Os construtores das subclasses devem utilizar **super** para delegar a inicialização dos atributos herdados da superclasse. O construtor da classe **caule** deve inicializar os atributos **comprimento** e **numeroFolhasCaule**, enquanto os construtores das subclasses devem inicializar seus próprios atributos específicos e das superclasses.

Identificador	Retorno	Parâmetro	Descrição
<code>crescimentoPotencial</code>	<code>void</code>	–	Calcula o crescimento potencial do caule
<code>crescimentoReal</code>	<code>void</code>	–	Calcula o crescimento real obtido
<code>abscisao</code>	<code>void</code>	–	Simula o processo de queda do órgão

Dica

Nos métodos faça apenas mensagens de saída (`System.out.println()`) indicando que o método foi chamado, sem implementar a lógica real de crescimento ou abscisão.

Classe Principal

Desenvolva uma classe **Principal** para instanciar um objeto do tipo **CaulePrincipal** e um do tipo **RamoCaule**, atribuindo valores iniciais por meio de construtores com **super**, testando os métodos de crescimento e exibindo as informações detalhadas de cada um por meio do método `toString()`.